

Д31: Знакомство с Core ML & Create ML

Курс: Разработка интеллектуальных приложений **Автор:** Nick **Задание:** установить `coremltools`, сконвертировать простую модель из Hugging Face / PyTorch Hub и запустить её в демо-проекте iOS.

В качестве модели взята **MobileNetV2** (эталонная реализация классификатора изображений из `torchvision` — это и есть PyTorch Hub). Она конвертируется в Core ML и запускается в iOS-приложении через Vision + `VNCoreMLRequest`. В конце приведён альтернативный вариант с моделью из Hugging Face.

Часть 1. Теория (кратко)

Два «кита» Apple ML:

- **Core ML** — инференс на устройстве. Формат моделей: `.mlmodel` (legacy NeuralNetwork) или `.mlpackage` (современный ML Program, по умолчанию с `coremltools` 5+).
- **Create ML** — обучение и конвертация без кода (шаблоны Image Classification, Sound, Text и т.д.).

Фреймворки-помощники:

- **Vision** — изображения (классификация, детекция, лица, текст). Даёт удобную обёртку `VNCoreMLRequest`, которая сама масштабирует и нормализует картинку под вход модели.
- **NaturalLanguage** — текст (язык, тональность, NER).

Где это в системе: Face ID (Secure Enclave), Live Text (Vision), Siri (NLP на устройстве).

Полный цикл интеграции ML: обучение/выбор модели → конвертация в Core ML → перетаскивание в Xcode (генерируется Swift-класс) → инференс через Vision/Core ML.

Часть 2. Настройка среды

```
# 1. Python 3.9–3.11 (coremltools пока не любит слишком свежие версии)
python3 --version

# 2. Виртуальное окружение (чтобы не засорять систему)
python3 -m venv .venv
source .venv/bin/activate

# 3. Зависимости
pip install --upgrade pip
pip install torch torchvision coremltools numpy
```

requirements.txt :

```
torch>=2.1.0
torchvision>=0.16.0
coremltools>=8.0
numpy
```

Требования: **Xcode 15+**, macOS. Python нужен только для конвертации — сам инференс на устройстве идёт средствами Core ML.

Часть 3. Конвертация модели (convert.py)

Ключевые моменты, которые есть в рабочем коде и которых нет в упрощённом примере из методички:

1. Модель нужно **перевести в eval-режим** и **трассировать** (`torch.jit.trace`) с примером входа — `ct.convert` не умеет брать «сырой» `.pth`.
2. Нормализацию ImageNet «вшиваем» прямо в модель через `scale / bias` в `ImageType` — тогда на стороне iOS не нужно вручную нормализовать пиксели.
3. Через `ClassifierConfig` передаём метки классов — тогда Vision вернёт готовые `VNClassificationObservation` с именами классов и вероятностями.

```
# convert.py – MobileNetV2 (PyTorch / torchvision) -> Core ML
import urllib.request
import numpy as np
import torch
import torchvision
import coremltools as ct

# 1. Предобученная модель (эталонная реализация из torchvision = PyTorch Hub)
weights = torchvision.models.MobileNet_V2_Weights.IMAGENET1K_V2
```

```

torch_model = torchvision.models.mobilenet_v2(weights=weights)
torch_model.eval()

# 2. Трассировка с примером входа нужной формы (NCHW, 224x224)
example_input = torch.rand(1, 3, 224, 224)
traced_model = torch.jit.trace(torch_model, example_input)

# 3. Метки классов ImageNet (1000 шт.)
label_url = "https://storage.googleapis.com/download.tensorflow.org/data/ImageNetLabels.txt"
class_labels = urllib.request.urlopen(label_url).read().decode("utf-8").splitlines()
class_labels = class_labels[1:] # первый класс – background, убираем
assert len(class_labels) == 1000

# 4. Предобработка ImageNet "вшивается" в модель.
# ImageType применяет: pixel * scale + bias (bias – по каналам).
mean = np.array([0.485, 0.456, 0.406])
std = np.array([0.229, 0.224, 0.225])
scale = 1.0 / (0.226 * 255.0) # усреднённый std (документация Apple)
bias = (-mean / std).tolist()

image_input = ct.ImageType(
    name="image",
    shape=example_input.shape,
    scale=scale,
    bias=bias,
)

# 5. Конвертация через Unified Conversion API
mlmodel = ct.convert(
    traced_model,
    inputs=[image_input],
    classifier_config=ct.ClassifierConfig(class_labels),
    minimum_deployment_target=ct.target.iOS16,
    compute_units=ct.ComputeUnit.ALL, # CPU + GPU + Neural Engine
)

# 6. Метаданные – их видно в Xcode при клике на модель
mlmodel.author = "Nick"
mlmodel.short_description = "MobileNetV2 image classifier (ImageNet, 1000 classes)"
mlmodel.input_description["image"] = "Входное изображение 224x224"

# 7. Сохранение. Современный формат – .mlpackage.
# Для legacy .mlmodel добавить в convert(): convert_to="neuralnetwork"
mlmodel.save("MobileNetV2.mlpackage")
print("Готово: MobileNetV2.mlpackage")

```

Запуск:

```

python convert.py
# -> в текущей папке появится MobileNetV2.mlpackage

```

Часть 4. Демо-проект iOS

Создать проект: **Xcode → New Project → App → Interface: SwiftUI**. Перетащить `MobileNetV2.mlpackage` в навигатор проекта (галочка *Copy items if needed*). Xcode автоматически сгенерирует Swift-класс `MobileNetV2`.

Deployment target: iOS 17+ (из-за нового `onChange` и `PhotosPicker`). В `Info.plist` доступ к галерее для `PhotosPicker` не требуется.

`ImageClassifier.swift`

```

import Vision
import CoreML
import UIKit

@MainActor
final class ImageClassifier: ObservableObject {
    @Published var results: [String] = []
    @Published var isBusy = false

    private let visionModel: VNCoreMLModel

    init() {
        let config = MLModelConfiguration()
        // MobileNetV2 – класс, сгенерированный Xcode из .mlpackage
        guard let coreMLModel = try? MobileNetV2(configuration: config).model,
              let vnModel = try? VNCoreMLModel(for: coreMLModel) else {
            fatalError("Не удалось загрузить модель MobileNetV2")
        }
        self.visionModel = vnModel
    }

    func classify(_ image: UIImage) {
        guard let cgImage = image.cgImage else { return }
        isBusy = true
        results = []

        let request = VNCoreMLRequest(model: visionModel) { [weak self] request, _ in
            Task { @MainActor in
                self?.isBusy = false
                guard let obs = request.results as? [VNClassificationObservation] else { return }
            }
        }
    }

```

```

        // топ-3 предсказания
        self?.results = obs.prefix(3).map {
            "\($0.identifier) — \($0.confidence * 100)%"
        }
    }
}
request.imageCropAndScaleOption = .centerCrop

let handler = VNImageRequestHandler(cgImage: cgImage, orientation: .up)
DispatchQueue.global(qos: .userInitiated).async {
    try? handler.perform([request])
}
}
}
}

```

ContentView.swift

```

import SwiftUI
import PhotosUI

struct ContentView: View {
    @StateObject private var classifier = ImageClassifier()
    @State private var pickerItem: PhotosPickerItem?
    @State private var image: UIImage?

    var body: some View {
        VStack(spacing: 20) {
            Group {
                if let image {
                    Image(uiImage: image)
                        .resizable()
                        .scaledToFit()
                } else {
                    RoundedRectangle(cornerRadius: 12)
                        .fill(.gray.opacity(0.15))
                        .overlay(Text("Выберите изображение").foregroundColor(.secondary))
                }
            }
            .frame(height: 300)
            .cornerRadius(12)

            PhotosPicker("Выбрать из галереи", selection: $pickerItem, matching: .images)
                .buttonStyle(.borderedProminent)

            if classifier.isBusy {
                ProgressView()
            } else {
                ForEach(classifier.results, id: \.self) { line in
                    Text(line).font(.headline)
                }
            }
            Spacer()
        }
        .padding()
        .onChange(of: pickerItem) { _, newItem in
            Task {
                guard let data = try? await newItem?.loadTransferable(type: Data.self),
                      let uiImage = UIImage(data: data) else { return }
                image = uiImage
                classifier.classify(uiImage)
            }
        }
    }
}

```

CoreMLDemoApp.swift

```

import SwiftUI

@main
struct CoreMLDemoApp: App {
    var body: some Scene {
        WindowGroup { ContentView() }
    }
}

```

Как работает: пользователь выбирает фото из галереи → UIImage уходит в ImageClassifier → Vision сам масштабирует картинку под вход 224×224 и запускает VNCoreMLRequest → на экран выводятся топ-3 класса с вероятностями.

Часть 5. Проверка результата

1. Запустить приложение на симуляторе или устройстве (кнопка ► в Xcode).
2. Выбрать из галереи фото объекта (кот, собака, чашка кофе, ноутбук).
3. Приложение показывает распознанный класс и уверенность, напр.: Egyptian cat — 87%.

Замена модели проверяется в одну строку: сконвертировать другую модель, перетащить новый .mlpackage, поменять имя класса в ImageClassifier (MobileNetV2(...) → новый класс). Логика Vision остаётся прежней.

Часть 6. Вариант с Hugging Face (опционально)

Задание допускает модель из Hugging Face. Пример — классификатор тональности текста (NaturalLanguage -сценарий); конвертируется по той же схеме через трассировку.

```
# convert_hf.py - DistilBERT sentiment (Hugging Face) -> Core ML
import torch, coremltools as ct
from transformers import AutoTokenizer, AutoModelForSequenceClassification

name = "distilbert-base-uncased-finetuned-sst-2-english"
tok = AutoTokenizer.from_pretrained(name)
model = AutoModelForSequenceClassification.from_pretrained(name, torchscript=True).eval()

enc = tok("A great movie", return_tensors="pt", padding="max_length", max_length=64)
example = (enc["input_ids"], enc["attention_mask"])
traced = torch.jit.trace(model, example)

mlmodel = ct.convert(
    traced,
    inputs=[
        ct.TensorType(name="input_ids", shape=example[0].shape, dtype=int),
        ct.TensorType(name="attention_mask", shape=example[1].shape, dtype=int),
    ],
    minimum_deployment_target=ct.target.iOS16,
)
mlmodel.save("Sentiment.mlpackage")
```

Часть 7. Выводы: готовая модель или кастомная?

Использовать готовую (Core ML)	Создавать кастомную (Create ML / Python)
Задача типовая: классификация изображений/текста	Задача уникальна для продукта (например, дефекты именно вашего товара)
Нужен быстрый старт и прототип	Есть размеченные данные (100-1000+ примеров на класс)
Точность SOTA-модели (Apple Model Gallery) устраивает	Готовые модели не подходят по классам или точности
Не хватает данных / ресурсов на обучение	Нужен полный контроль над архитектурой

Главный вывод. Экосистема Apple ML демократизирует машинное обучение: основной инструмент — Xcode (Core ML + Vision + Create ML), а Python с coremltools нужен только для конвертации сложных моделей из PyTorch/TensorFlow/Hugging Face. Ценность разработчика — не в написании алгоритмов с нуля, а в правильном выборе инструмента и грамотной интеграции ML в решение реальной пользовательской задачи.